

BASES DE DATOS II

Base de Datos II

Trabajo Práctico — Parcial 2



StreamBase — MVP con MongoDB

Tecnicatura Universitaria en Programación | Div133



mongoDB

N



¿Qué hace este programa?

StreamBase es un MVP (Mínimo Producto Viable) que simula el backend de una plataforma de streaming tipo Netflix. Demuestra el modelado de datos en MongoDB usando decisiones fundamentadas de embedding y referencing, y ejecuta tres Aggregation Pipelines reales sobre la base de datos.



GRUPO 14

Sebastián Lescano
Joaquín Avallone
Lautaro Subeldia
Thiago Martínez
Mateo Corbetto



MODELADO DE DATOS

Embedding y Referencing



AGGREGATION

3 Pipelines reales de consulta



MONGODB

Colecciones y relaciones



MVP FUNCIONAL

Backend de una plataforma de streaming



PYTHON
3.9 o superior



MONGODB
localhost:27017



PYTHON
seed.py
queries.py



GITHUB
LautaroSantiago/
streambase-bd2



AÑO
2026

PARTE 2 — Modelado NoSQL: Diseño y Justificación

1. Criterio de diseño: primero las consultas, después el modelo

Antes de definir cualquier colección, identificamos las operaciones más frecuentes del sistema. En MongoDB, el diseño no parte de la normalización sino de las consultas: se modela en función de cómo se va a leer y escribir la información. Este principio, conocido como *diseño impulsado por consultas*, fue el marco que guió todas nuestras decisiones.

Las consultas principales que el sistema debe responder eficientemente son:

1. Mostrar el catálogo con título, géneros, director y calificación de cada título.
2. Reproducir contenido y registrar el progreso por perfil (retomar donde se dejó).
3. Consultar el historial de visualización de un perfil específico.
4. Mostrar la lista de contenido guardado ("Mi Lista") de un perfil.
5. Filtrar series por temporada y episodio para continuar la reproducción.

Con este mapa de lecturas y escrituras definido, procedemos al diseño.

2. Entidades del sistema y sus relaciones

El sistema de streaming cuenta con las siguientes entidades principales:

Entidad	Descripción
cuentas	Cuenta del suscriptor, con plan de suscripción
perfiles	Hasta 5 perfiles por cuenta (como en Netflix)
contenido	Catálogo unificado de películas y series
temporadas	Temporadas de una serie con sus episodios
reproducciones	Historial de visualización por perfil
mi_lista	Contenido guardado por cada perfil

Las relaciones entre entidades son:

- cuentas 1 → N perfiles: una cuenta puede tener hasta 5 perfiles.
- contenido 1 → N temporadas: una serie puede tener muchas temporadas.
- temporadas 1 → N episodios: cada temporada contiene sus episodios (embebidos).
- perfiles N ↔ M contenido vía reproducciones: un perfil puede ver muchos títulos, un título puede ser visto por muchos perfiles. Relación sin límites.
- perfiles N ↔ M contenido vía mi_lista: un perfil guarda muchos títulos en su lista.

3. Colecciones, documentos de ejemplo y decisiones de diseño

3.1 Colección contenido

Es la colección central del catálogo. Unifica películas y series en un único tipo de documento usando el campo discriminador `tipo`. Esta decisión de **polimorfismo documental** es una ventaja directa del modelo NoSQL: en un esquema relacional hubiéramos necesitado dos tablas separadas o una tabla con muchas columnas nulas.

PARTE 3 — Resumen Teórico: El Aggregation Pipeline de MongoDB

1. ¿Qué es y por qué existe?

Cuando trabajamos con una base de datos, hay dos tipos de operaciones bien distintas: buscar un documento y transformar un conjunto de documentos para obtener información derivada. `find()` resuelve el primer caso. Para el segundo existe el **Aggregation Pipeline**.

El Aggregation Pipeline es el motor de procesamiento y análisis de datos de MongoDB. Permite tomar una colección de documentos y hacerlos pasar por una secuencia de etapas (stages), donde cada etapa transforma la salida de la anterior. La metáfora es la de una línea de montaje industrial: los documentos entran por un extremo, cada estación hace algo con ellos, y al final del recorrido obtenemos un resultado completamente diferente al input original.

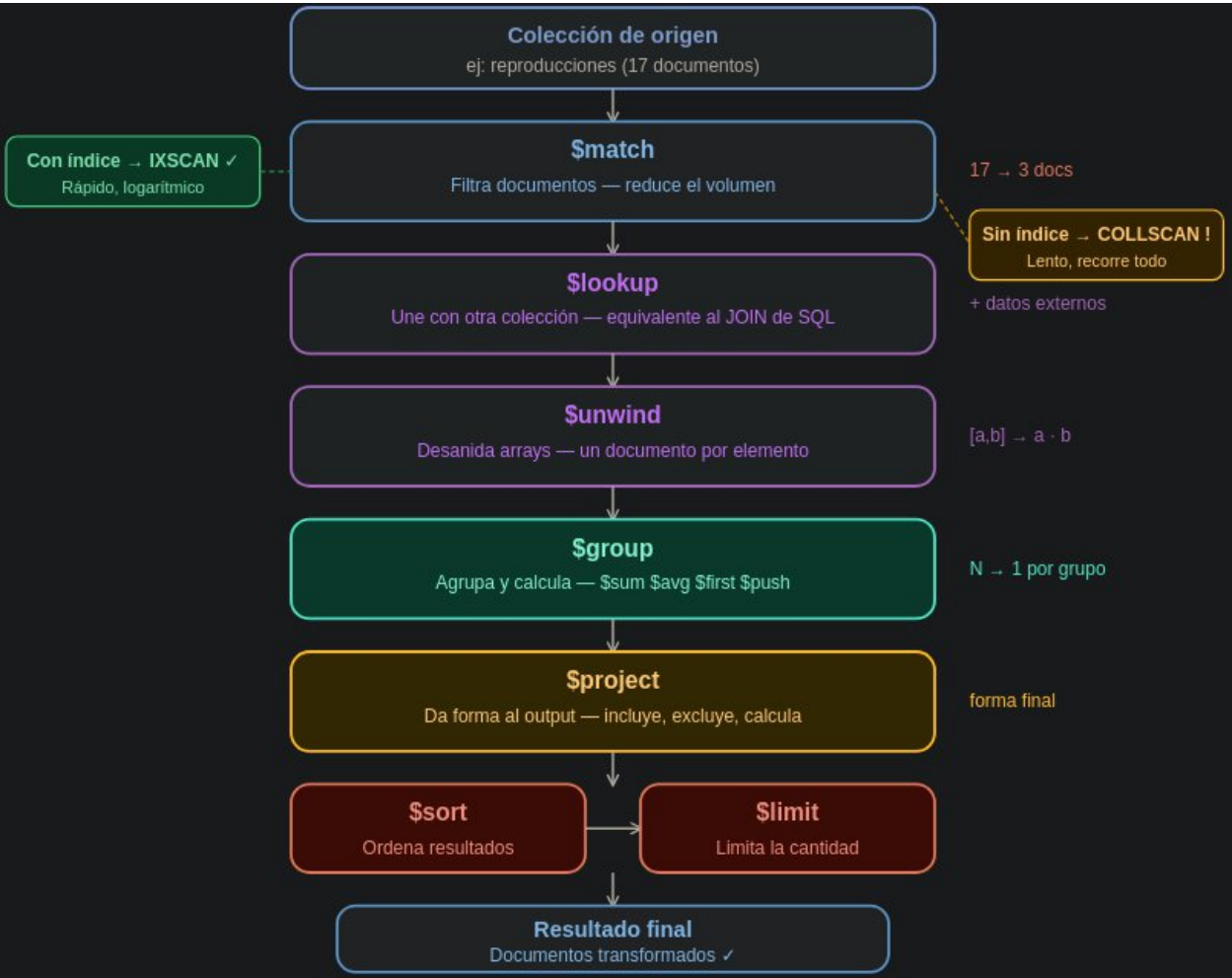
La diferencia conceptual clave con `find()` es la siguiente:

	<code>find()</code>	Aggregation Pipeline
¿Qué hace?	Filtra y proyecta documentos existentes	Transforma, calcula y combina documentos
Resultado	Documentos de la colección (tal cual o con proyección)	Documentos nuevos, calculados, agrupados
Cuándo usarlo	"Dame los usuarios con plan premium"	"Dame cuántas reproducciones tuvo cada película este mes"
Equivalente SQL	SELECT ... WHERE	SELECT ... GROUP BY ... HAVING ... JOIN

En términos prácticos: si necesitamos contar, sumar, promediar, agrupar, combinar colecciones o calcular campos derivados, usamos el Pipeline. Si solo necesitamos recuperar documentos que cumplan una condición, usamos `find()`.

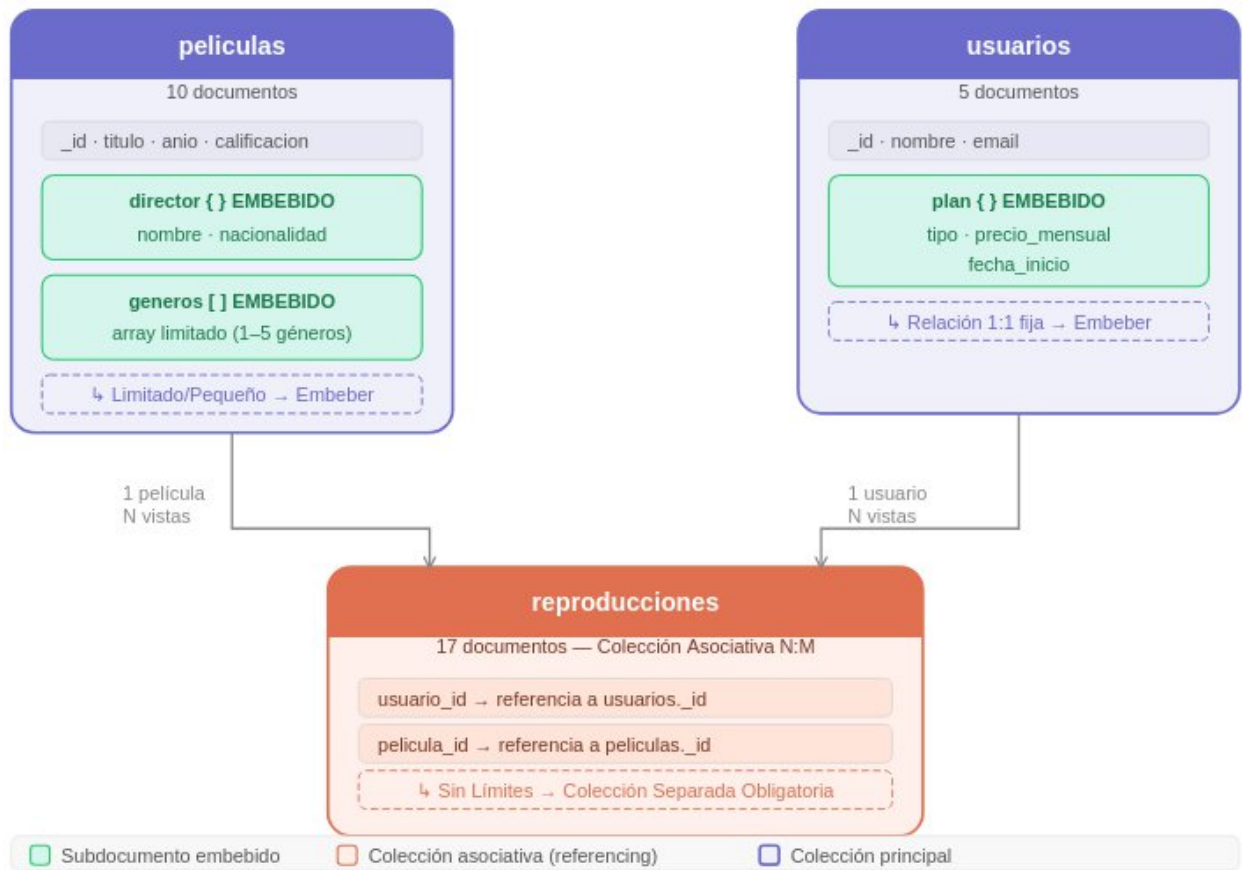
2. Arquitectura del Pipeline: cómo fluyen los documentos

El pipeline se define como un array de objetos. Cada objeto es una etapa identificada por un operador que empieza con `$`. MongoDB ejecuta las etapas en orden, pasando la salida de cada una como entrada de la siguiente:



Este orden es el recomendado para la mayoría de los casos. No es obligatorio usarlas todas ni en ese orden exacto, pero respetarlo tiene implicancias de rendimiento importantes (ver sección 4).

Diagrama de Modelo del MVP 3: colecciones



3. Las etapas principales: qué hace cada una

\$match — Filtrar

Filtra los documentos que pasan a la siguiente etapa. Funciona igual que el filtro de `find()`. Es la etapa más importante desde el punto de vista del rendimiento: debe ir lo antes posible en el pipeline para reducir el volumen de datos que procesan las etapas siguientes.

Ejemplo de nuestro proyecto: filtrar solo las reproducciones de un usuario

```
{ "$match": { "usuario_id": 1 } }
```

Si el campo `usuario_id` tiene un índice, MongoDB usará un **IXSCAN** (recorrido por índice) en lugar de un **COLLSCAN** (recorrido completo de la colección). La diferencia de rendimiento puede ser de órdenes de magnitud en colecciones grandes.

\$lookup — Unir colecciones (el JOIN de MongoDB)

Trae documentos de otra colección y los agrega como un array al documento actual. Es el equivalente al JOIN de SQL, pero orientado a documentos.

Une cada reproducción con los datos de la película correspondiente

```
{
```

```

"$lookup": {
  "from": "peliculas",    # colección con la que unimos
  "localField": "pelicula_id", # campo en la colección actual
  "foreignField": "_id",    # campo en la colección remota
  "as": "info_pelicula"    # nombre del array resultado
}
}

```

El resultado de `$lookup` es siempre un **array**, aunque haya un solo elemento coincidente. Por eso casi siempre va seguido de `$unwind`.

Consideración de diseño importante: si necesitamos hacer `$lookup` frecuentemente en lecturas de alta velocidad, es una señal de que podría convenir embeber ese dato. En nuestro modelo, usamos `$lookup` justamente para las relaciones que elegimos referenciar (porque crecen sin límite). Es una consecuencia esperada y aceptada del diseño.

`$unwind` — Desanidar arrays

Toma un campo que es un array y genera un documento separado por cada elemento del array. Es esencial después de `$lookup` (que siempre devuelve un array) y también para trabajar con arrays embebidos como `generos`.

Convierte el array "info_pelicula" (con 1 elemento) en un campo plano

```
{ "$unwind": "$info_pelicula" }
```

También se usa para desanidar el array de géneros:

Un documento con `generos`: ["Drama", "Acción"] se convierte en

dos documentos: uno con `genero`: "Drama" y otro con `genero`: "Acción"

```
{ "$unwind": "$pelicula.generos" }
```

Antes del `$unwind`:

```
{ "pelicula_id": 1, "info_pelicula": [{ "titulo": "Inception", "calificacion": 8.8 }] }
```

Después del `$unwind`:

```
{ "pelicula_id": 1, "info_pelicula": { "titulo": "Inception", "calificacion": 8.8 } }
```

`$group` — Agrupar y calcular

Es la etapa más poderosa del pipeline. Agrupa documentos por un campo (o expresión) y aplica funciones de acumulación sobre el grupo.

El campo `_id` de `$group` define el criterio de agrupación. Si `_id: null`, agrupa todos los documentos juntos (útil para totales globales).

Contar cuántas veces fue vista cada película

```
{
  "$group": {
    "_id": "$pelicula_id",          # agrupar por película
    "total_reproducciones": { "$sum": 1 }, # contar
    "minutos_totales": { "$sum": "$duracion_min" } # sumar campo
  }
}
```

Operadores de acumulación disponibles:

Operador	Qué calcula	Ejemplo
<code>\$sum</code>	Suma (o cuenta si es 1)	<code>{ "\$sum": "\$duracion_min" }</code>
<code>\$avg</code>	Promedio	<code>{ "\$avg": "\$calificacion" }</code>
<code>\$max / \$min</code>	Máximo / Mínimo	<code>{ "\$max": "\$fecha" }</code>
<code>\$first / \$last</code>	Primer/último valor del grupo	<code>{ "\$first": "\$usuario.nombre" }</code>
<code>\$push</code>	Agrega valores a un array	<code>{ "\$push": "\$titulo" }</code>
<code>\$addToSet</code>	Como <code>\$push</code> pero sin duplicados	<code>{ "\$addToSet": "\$genero" }</code>

\$project — Dar forma al output

Decide qué campos aparecen en el resultado final, cómo se llaman y si se calculan nuevos campos. Es el equivalente al `SELECT` de SQL.

```
{
  "$project": {
    "_id": 0,          # excluir el _id
    "titulo": "$info_pelicula.titulo", # renombrar y tomar de subdocumento
    "total_reproducciones": 1,      # incluir tal cual
    "total_horas": {                # campo calculado
```



```

    "$round": [{ "$divide": ["$total_minutos", 60] }, 2]
  }
}
}

```

Regla: dentro de `$project`, 1 = incluir, 0 = excluir. No se pueden mezclar inclusiones y exclusiones en el mismo `$project` (salvo para `_id`).

\$sort — Ordenar

Ordena los documentos por uno o más campos. 1 es ascendente, -1 es descendente.

Ordenar de mayor a menor cantidad de reproducciones

```
{ "$sort": { "total_reproducciones": -1 } }
```

Si `$sort` va **antes** de `$group`, puede usar índices. Si va después, ordena en memoria. Para datasets grandes, es importante ubicarlo estratégicamente.

\$limit — Limitar resultados

Pasa solo los primeros N documentos a la siguiente etapa. Siempre debe ir después de `$sort` para que el resultado sea determinístico.

Tomar solo los 5 primeros

```
{ "$limit": 5 }
```

La combinación `$sort + $limit` es el patrón estándar para rankings y top-N.

4. Reglas de rendimiento: cómo escribir un pipeline eficiente

Regla 1: \$match lo antes posible

Cada etapa procesa todos los documentos que recibe. Si ponemos `$match` al principio, las etapas siguientes trabajan con menos documentos. Si lo ponemos al final, hemos procesado datos que íbamos a descartar.

Correcto: filtramos primero, después hacemos el `$lookup` costoso

```

pipeline = [
  { "$match": { "usuario_id": 1 } }, # ← aquí filtramos
  { "$lookup": { ... } },          # solo une los docs que pasaron
  { "$group": { ... } }
]

```

× Incorrecto: hacemos `$lookup` sobre toda la colección y luego filtramos

```
pipeline = [
  { "$lookup": { ... } },          # une TODOS los documentos
  { "$match": { "usuario_id": 1 } } # recién ahora filtra
]
```

Regla 2: aprovechar índices con \$match al inicio

Si el campo del \$match tiene un índice, MongoDB ejecuta un **IXSCAN** (busca por índice, muy rápido). Si no hay índice o \$match no está al principio, hace un **COLLSCAN** (recorre toda la colección). En la colección **reproducciones**, tener un índice sobre **usuario_id** y **pelicula_id** es fundamental para el rendimiento.

Regla 3: \$project para reducir el peso de los documentos

Si los documentos son grandes, un \$project temprano que elimine campos innecesarios reduce la memoria que usan las etapas siguientes.

5. Los tres pipelines del proyecto StreamBase

Pipeline 1 — Top 5 películas más vistas

Qué resuelve: ranking de contenido por popularidad. Fundamental para la pantalla de inicio de cualquier plataforma de streaming.

```
pipeline = [
  { "$group": {
    "_id": "$pelicula_id",
    "total_reproducciones": { "$sum": 1 }
  } },
  { "$sort": { "total_reproducciones": -1 } },
  { "$limit": 5 },
  { "$lookup": {
    "from": "peliculas",
    "localField": "_id",
    "foreignField": "_id",
    "as": "info_pelicula"
  } },
  { "$unwind": "$info_pelicula" },
```



```
{ "$project": {
  "_id": 0,
  "titulo": "$info_pelicula.titulo",
  "calificacion": "$info_pelicula.calificacion",
  "total_reproducciones": 1
}}
]
```

Análisis de las decisiones: el *'lookup' va*después*del 'lookup' va*después*del 'limit'* deliberadamente. Si lo pusiéramos antes, haríamos el join sobre toda la colección. Al limitarlo a 5 primero, solo unimos 5 documentos con películas. Esto es un ejemplo de optimización por orden de etapas.

Resultado con nuestros datos:

1. Inception → 3 reproducciones ☆ 8.8
2. The Dark Knight → 2 reproducciones ☆ 9.0
3. Avengers Endgame → 2 reproducciones ☆ 8.4
4. Interstellar → 2 reproducciones ☆ 8.6
5. The Matrix → 2 reproducciones ☆ 8.7

Pipeline 2 — Géneros favoritos de un usuario

Qué resuelve: personalización. Saber qué géneros prefiere un usuario permite mostrarle recomendaciones relevantes.

```
pipeline = [
  { "$match": { "usuario_id": 1 } },    # Sebastián Lescano
  { "$lookup": {
    "from": "peliculas",
    "localField": "pelicula_id",
    "foreignField": "_id",
    "as": "pelicula"
  }},
  { "$unwind": "$pelicula" },
```

```

{ "$unwind": "$pelicula.generos" },    # un doc por cada género

{ "$group": {

  "_id": "$pelicula.generos",

  "cantidad": { "$sum": 1 }

}},

{ "$sort": { "cantidad": -1 } },

{ "$project": {

  "_id": 0,

  "genero": "$_id",

  "cantidad": 1

}} ]

```

Análisis: este pipeline usa \$unwind dos veces. La primera desanida el array del \$lookup. La segunda desanida el array **generos** embebido en cada película, generando un documento por género por reproducción. Después, \$group cuenta cuántas veces aparece cada género. Es un patrón clásico para analizar datos anidados.

Resultado con nuestros datos (usuario Sebastián, que vio Inception, The Dark Knight e Interstellar):

Ciencia Ficción → 2 Acción → 2 Drama → 2

Thriller → 1 Crimen → 1 Aventura → 1

Pipeline 3 — Usuarios premium con más tiempo reproducido

Qué resuelve: segmentación de usuarios. Permite identificar los suscriptores premium más comprometidos para campañas de retención.

```

pipeline = [

  { "$lookup": {

    "from": "usuarios",

    "localField": "usuario_id",

    "foreignField": "_id",

    "as": "usuario"

  }},

  { "$unwind": "$usuario" },

  { "$match": { "usuario.plan.tipo": "premium" } }, # notación de punto

  { "$group": {

```

```

    "_id": "$usuario_id",

    "nombre": { "$first": "$usuario.nombre" },

    "total_minutos": { "$sum": "$duracion_min" }

  }},

  { "$sort": { "total_minutos": -1 } },

  { "$project": {

    "_id": 0,

    "nombre": 1,

    "total_minutos": 1,

    "total_horas": { "$round": [{ "$divide": ["$total_minutos", 60] }, 2] }

  }} ]

```

Análisis: el `$match` está después del `$lookup` porque necesitamos los datos del usuario para filtrar por `plan`. No podemos filtrar antes porque el campo `plan.tipo` no existe en `reproducciones`. Aquí la **notación de punto** (`usuario.plan.tipo`) accede a un subdocumento embebido, el mismo patrón visto en clase con `direccion.calle`.

Resultado con nuestros datos:

1. Joaquín Avallone → 597 min (9.95 hs)
2. Thiago Martínez → 570 min (9.5 hs)
3. Sebastián Lescano → 412 min (6.87 hs)

6. Aggregation Pipeline vs. código de aplicación

Una pregunta frecuente es: ¿por qué hacer esto en el pipeline y no en Python después de traer los datos con `find()`?

La respuesta es el **principio de localidad del procesamiento**: es mucho más eficiente transformar los datos donde están almacenados (en MongoDB) que traer miles de documentos a la aplicación y procesarlos en memoria.

Ejemplo concreto: el Pipeline 1 agrupa 17 documentos en MongoDB y devuelve 5 resultados. Si lo hiciéramos en Python:

1. Traemos los 17 documentos con `find()`
2. Hacemos un bucle para contar por `pelicula_id`
3. Ordenamos el resultado
4. Tomamos los primeros 5
5. Hacemos una segunda consulta a `peliculas` para enriquecer los datos

Con el pipeline, todo eso ocurre en el motor de MongoDB en una sola operación. En colecciones con millones de documentos, la diferencia es crucial.

7. Cuándo NO usar el Aggregation Pipeline

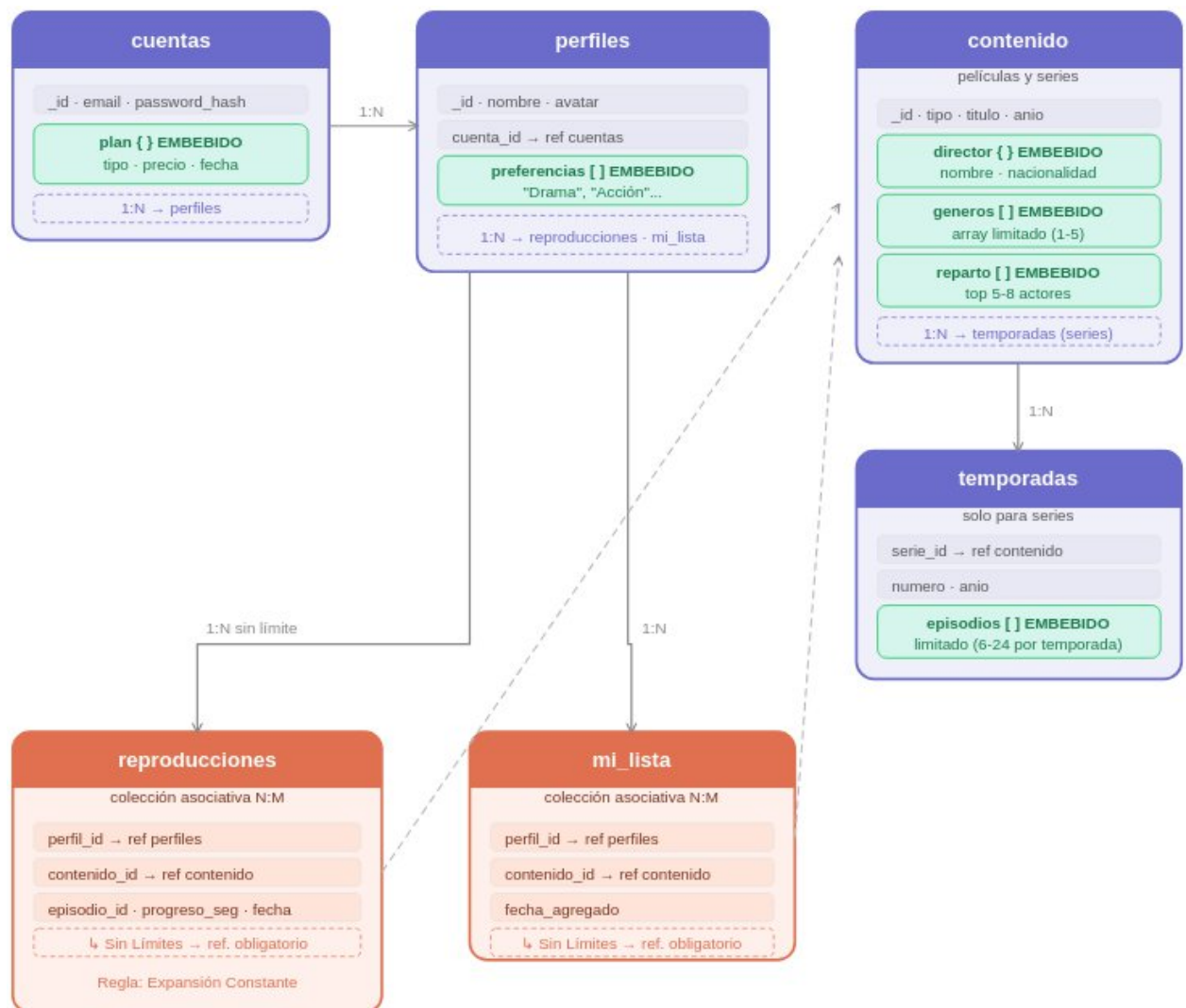
El pipeline tiene un costo de escritura (el código es más verboso y más difícil de leer que un `find()`).

No tiene sentido usarlo cuando:

- Solo necesitamos filtrar y proyectar sin calcular nada → usar `find()`.
- La lógica de transformación es muy simple y los datos son pocos → hacerlo en la aplicación es más claro.
- El pipeline resultaría más complejo que el beneficio de rendimiento que aporta.

La regla práctica: si el resultado requiere un `GROUP BY`, un `JOIN` o un campo calculado, el pipeline es la herramienta correcta.

Diagrama de Modelo conceptual completo: 6 colecciones



PARTE 4 — Cheatsheet de Python orientado a MongoDB y pymongo

Nota de adaptación: La consigna original solicita una cheatsheet de JavaScript/Node.js. Dado que el lenguaje elegido para el proyecto es Python, esta hoja de referencia está orientada a Python con el driver oficial pymongo, cubriendo los mismos conceptos pero en el ecosistema correspondiente. Se incluyen patrones y funcionalidades que van más allá del contenido visto en clase.

1. Conexión y gestión de la base de datos

Conexión básica con variables de entorno

```
import os

from pymongo import MongoClient

from dotenv import load_dotenv

load_dotenv() # carga el archivo .env

client = MongoClient(os.getenv("MONGO_URI", "mongodb://localhost:27017"))

db = client["streambase"]

# Siempre cerrar la conexión al terminar

client.close()
```

Context manager — conexión segura (patrón recomendado)

```
# MongoClient implementa el protocolo de context manager

# La conexión se cierra automáticamente al salir del bloque

with MongoClient(os.getenv("MONGO_URI")) as client:

    db = client["streambase"]

    resultado = db.peliculas.find_one({ "titulo": "Inception" })

# client.close() se llama automáticamente aquí
```

Verificar la conexión antes de operar

```
try:

    client.admin.command("ping")

    print("  Conectado a MongoDB")

except Exception as e:

    print(f"× Error de conexión: {e}")
```

2. CRUD — Operaciones esenciales

Insertar

Un documento

```
db.peliculas.insert_one({ "titulo": "Inception", "anio": 2010 })
```

Varios documentos — devuelve InsertManyResult

```
resultado = db.peliculas.insert_many([  
    { "titulo": "Parasite", "anio": 2019 },  
    { "titulo": "The Matrix", "anio": 1999 }  
])
```

```
print(resultado.inserted_ids) # lista de ObjectIds generados
```

Consultar

Un documento (devuelve dict o None)

```
pelicula = db.peliculas.find_one({ "titulo": "Inception" })
```

Varios — find() devuelve un Cursor (iterable, NO una lista)

```
cursor = db.peliculas.find({ "calificacion": { "$gte": 8.5 } })
```

Convertir cursor a lista — cargamos todo en memoria

```
peliculas = list(cursor)
```

Proyección — segundo argumento de find()

```
db.peliculas.find({}, { "titulo": 1, "calificacion": 1, "_id": 0 })
```

Encadenar operaciones sobre el cursor (antes de iterar)

```
db.peliculas.find().sort("calificacion", -1).limit(5).skip(0)
```

Contar documentos

```
total = db.peliculas.count_documents({ "anio": { "$gte": 2010 } })
```


Valores únicos de un campo

```
generos_unicos = db.peliculas.distinct("generos")
```

Actualizar

Actualizar un documento

```
db.usuarios.update_one(
    { "_id": 1 },          # filtro
    { "$set": { "plan.tipo": "premium" } }  # modificación (notación de punto)
)
```

Actualizar varios

```
db.peliculas.update_many(
    { "anio": { "$lt": 2000 } },
    { "$set": { "era": "clásica" } }
)
```

upsert=True: inserta si no existe, actualiza si existe

```
db.peliculas.update_one(
    { "titulo": "Nuevo Film" },
    { "$set": { "calificacion": 7.5 } },
    upsert=True
)
```

Eliminar

```
db.peliculas.delete_one({ "_id": 1 })
```

```
db.peliculas.delete_many({ "calificacion": { "$lt": 5.0 } })
```

3. Aggregation Pipeline

Estructura base en Python

```
pipeline = [
    { "$match": { ... } },
    { "$lookup": { "from": "...", "localField": "...", "foreignField": "...", "as": "..." } },
    { "$unwind": "$campo" },
```

```
{ "$group": { "_id": "$campo", "total": { "$sum": 1 } } },
{ "$sort": { "total": -1 } },
{ "$limit": 10 },
{ "$project": { "_id": 0, "campo": 1 } }
]
```

```
resultados = list(db.coleccion.aggregate(pipeline))
```

Operadores de acumulación en \$group

```
{ "$group": {
  "_id": "$categoria",
  "cantidad": { "$sum": 1 },          # contar
  "suma_minutos": { "$sum": "$duracion_min" }, # sumar campo
  "promedio": { "$avg": "$calificacion" }, # promedio
  "maximo": { "$max": "$calificacion" }, # máximo
  "primero": { "$first": "$titulo" }, # primer valor del grupo
  "lista_titulos": { "$push": "$titulo" } # acumular en array
}}
```

Campos calculados en \$project

```
{ "$project": {
  "titulo": 1,
  "horas": { "$divide": ["$duracion_min", 60] }, # división
  "horas_redondeadas": { "$round": [{ "$divide": ["$duracion_min", 60] }, 2] },
  "es_reciente": { "$gte": ["$anio", 2015] }, # booleano calculado
  "titulo_upper": { "$toUpper": "$titulo" } # transformación de string
}}
```

4. Procesamiento de resultados con Python

List comprehensions — equivalente a map() y filter()

```
peliculas = list(db.peliculas.find())
```

```
# Equivalente a map(): extraer solo los títulos
```

```
titulos = [p["titulo"] for p in peliculas]
```

```
# Equivalente a filter(): solo películas recientes
```

```
recientes = [p for p in peliculas if p["anio"] >= 2010]
```

```
# Combinado — títulos de películas recientes con buena calificación
```

```
seleccion = [  
    p["titulo"]  
    for p in peliculas  
    if p["anio"] >= 2010 and p["calificacion"] >= 8.5  
]
```

```
# Transformar estructura (similar a map con lógica)
```

```
resumen = [  
    { "titulo": p["titulo"], "director": p["director"]["nombre"] }  
    for p in peliculas  
]
```

map() y filter() funcionales

```
# map() — aplica una función a cada elemento
```

```
titulos = list(map(lambda p: p["titulo"], peliculas))
```

```
# filter() — filtra elementos según condición
```

```
premium = list(filter(lambda u: u["plan"]["tipo"] == "premium", usuarios))
```

```
# Encadenados
```

```
resultado = list(map(  
    lambda p: p["titulo"],  
    filter(lambda p: p["calificacion"] >= 9.0, peliculas)  
))
```

reduce() — para acumulaciones

```
from functools import reduce
```

```
# Sumar todos los minutos de reproducciones
```

```
total_minutos = reduce(
    lambda acc, r: acc + r["duracion_min"],
    reproducciones,
    0 # valor inicial
)
```

sorted() con clave personalizada

```
# Ordenar por calificación descendente
```

```
por_calificacion = sorted(peliculas, key=lambda p: p["calificacion"], reverse=True)
```

```
# Ordenar por múltiples campos (año descendente, luego título ascendente)
```

```
ordenadas = sorted(peliculas, key=lambda p: (-p["anio"], p["titulo"]))
```

zip() y enumerate() para procesar resultados

```
# enumerate() — agregar índice al iterar
```

```
for i, pelicula in enumerate(peliculas, start=1):
```

```
    print(f"{i}. {pelicula['titulo']}")
```

```
# zip() — combinar dos listas paralelas
```

```
ids = [1, 2, 3]
```

```
titulos = ["Inception", "Parasite", "The Matrix"]
```

```
for id_, titulo in zip(ids, titulos):
```

```
    print(f"ID {id_}: {titulo}")
```

5. Diccionarios — operaciones útiles con documentos MongoDB

Unpacking y merge de diccionarios

```
# Crear un documento base y agregar campos dinámicamente
```

```
base = { "titulo": "Inception", "anio": 2010 }
```

```
extra = { "calificacion": 8.8, "director": { "nombre": "Nolan" } }
```

Merge con ** (equivalente al spread operator de JS)

```
documento_completo = { **base, **extra }
```

```
# → { "titulo": "Inception", "anio": 2010, "calificacion": 8.8, ... }
```

Útil para construir pipelines dinámicamente

```
filtro_base = { "anio": { "$gte": 2000 } }
```

```
filtro_extra = { "calificacion": { "$gte": 8.0 } }
```

```
filtro_combinado = { **filtro_base, **filtro_extra }
```

Desestructuración (unpacking)

Extraer campos específicos de un documento

```
pelicula = db.peliculas.find_one({ "_id": 1 })
```

```
titulo, anio, calificacion = pelicula["titulo"], pelicula["anio"], pelicula["calificacion"]
```

Con .get() para campos opcionales (evita KeyError)

```
director = pelicula.get("director", {})
```

```
nombre_director = director.get("nombre", "Desconocido")
```

Forma compacta

```
nombre_director = pelicula.get("director", {}).get("nombre", "Desconocido")
```

Comprensión de diccionarios

Crear un dict { titulo: calificacion } desde una lista de documentos

```
calificaciones = { p["titulo"]: p["calificacion"] for p in peliculas }
```

```
# → { "Inception": 8.8, "The Dark Knight": 9.0, ... }
```

Filtrar claves de un documento (excluir _id y campos privados)

```
documento_limpio = { k: v for k, v in pelicula.items() if k != "_id" }
```

6. Manejo de errores — try/except en operaciones con MongoDB

Errores específicos de pymongo

```
from pymongo.errors import (
```

```
    ConnectionFailure,    # no se pudo conectar
```

```

DuplicateKeyError,    # violación de índice único

OperationFailure,     # error en operación de MongoDB

ServerSelectionTimeoutError # timeout al seleccionar servidor
)

```

```
def insertar_usuario(db, usuario):
```

```

    try:

        resultado = db.usuarios.insert_one(usuario)

        return resultado.inserted_id

    except DuplicateKeyError:

        print(f"× Ya existe un usuario con ese email.")

        return None

    except OperationFailure as e:

        print(f"× Error de operación: {e}")

        return None

```

```
def conectar():
```

```

    try:

        client = MongoClient(os.getenv("MONGO_URI"), serverSelectionTimeoutMS=3000)

        client.admin.command("ping")

        return client

    except ServerSelectionTimeoutError:

        print("× No se pudo conectar a MongoDB. ¿Está corriendo el servicio?")

        return None

    except ConnectionFailure as e:

        print(f"× Falla de conexión: {e}")

        return None

```

Patrón finally para cerrar siempre la conexión

```
client = None
```

```
try:
```

```

client = MongoClient(os.getenv("MONGO_URI"))

db = client["streambase"]

resultado = list(db.peliculas.find())

except Exception as e:

    print(f"Error: {e}")

finally:

    if client:

        client.close() # siempre se ejecuta, haya error o no

```

7. Módulos de Python útiles en el contexto del proyecto

os y python-dotenv — variables de entorno

```

import os

from dotenv import load_dotenv

load_dotenv() # lee el archivo .env del directorio actual

MONGO_URI = os.getenv("MONGO_URI")      # None si no existe

DB_NAME = os.getenv("DB_NAME", "default") # "default" como fallback

# Verificar que la variable existe antes de usarla

if not MONGO_URI:

    raise ValueError("MONGO_URI no está definida en el archivo .env")

```

pathlib — rutas de archivos de forma robusta

```

from pathlib import Path

# Definir ruta al .env relativa al script actual (más robusto que rutas relativas)

env_path = Path(__file__).parent / ".env"

load_dotenv(env_path)

# Leer un archivo JSON con datos de ejemplo

import json

```

```
data_path = Path(__file__).parent / "data" / "películas.json"
```

```
with data_path.open(encoding="utf-8") as f:
```

```
    películas = json.load(f)
```

pprint — mostrar documentos JSON legibles en consola

```
from pprint import pprint
```

```
película = db.películas.find_one({ "titulo": "Inception" })
```

```
pprint(película) # muestra el documento con indentación y saltos de línea
```

datetime — fechas en documentos MongoDB

```
from datetime import datetime, timezone
```

```
# MongoDB guarda fechas como objetos datetime (BSON Date)
```

```
ahora = datetime.now(timezone.utc)
```

```
db.reproducciones.insert_one({
```

```
    "usuario_id": 1,
```

```
    "película_id": 1,
```

```
    "fecha": ahora # pymongo lo convierte a ISODate automáticamente
```

```
})
```

```
# Filtrar por rango de fechas
```

```
from datetime import timedelta
```

```
hace_30_dias = datetime.now(timezone.utc) - timedelta(days=30)
```

```
db.reproducciones.find({ "fecha": { "$gte": hace_30_dias } })
```

8. Índices con pymongo

```
from pymongo import ASCENDING, DESCENDING, TEXT
```

```
# Índice simple
```

```
db.reproducciones.create_index("usuario_id")
```


Índice compuesto — útil para queries que filtran por los dos campos juntos

```
db.reproducciones.create_index([
    ("usuario_id", ASCENDING),
    ("pelicula_id", ASCENDING)
])
```

Índice único — garantiza que no haya duplicados

```
db.usuarios.create_index("email", unique=True)
```

Índice de texto — para búsquedas full-text

```
db.peliculas.create_index([("titulo", TEXT), ("sinopsis", TEXT)])
```

Ver índices de una colección

```
print(list(db.reproducciones.index_information().keys()))
```

9. Operaciones bulk — múltiples escrituras en una sola llamada

```
from pymongo import InsertOne, UpdateOne, DeleteOne
```

```
operaciones = [
```

```
    InsertOne({ "titulo": "Nuevo Film", "anio": 2025 }),
```

```
    UpdateOne({ "_id": 1 }, { "$set": { "calificacion": 9.1 } }),
```

```
    DeleteOne({ "_id": 99 })
```

```
]
```

```
resultado = db.peliculas.bulk_write(operaciones)
```

```
print(f"Insertados: {resultado.inserted_count}")
```

```
print(f"Actualizados: {resultado.modified_count}")
```

```
print(f"Eliminados: {resultado.deleted_count}")
```

10. Referencia rápida de operadores de consulta

Comparación

```
{ "calificacion": { "$eq": 8.8 } } # igual
```

```
{ "calificacion": { "$ne": 8.8 } } # distinto
{ "calificacion": { "$gt": 8.0 } } # mayor que
{ "calificacion": { "$gte": 8.0 } } # mayor o igual
{ "calificacion": { "$lt": 9.0 } } # menor que
{ "calificacion": { "$lte": 9.0 } } # menor o igual
{ "anio": { "$in": [2010, 2014, 2019] } } # en lista
{ "anio": { "$nin": [2010, 2014] } } # no en lista
```

Lógicos

```
{ "$and": [{ "anio": 2010 }, { "calificacion": { "$gte": 8.0 } } ] }
{ "$or": [{ "tipo": "pelicula" }, { "tipo": "serie" } ] }
{ "calificacion": { "$not": { "$lt": 8.0 } } }
```

Existencia y tipo

```
{ "sinopsis": { "$exists": True } } # el campo existe
{ "duracion_min": { "$exists": False } } # el campo no existe (ej: en series)
{ "anio": { "$type": "int" } } # tipo de dato BSON
```

Arrays

```
{ "generos": "Drama" } # contiene el elemento
{ "generos": { "$all": ["Drama", "Crimen"] } } # contiene todos
{ "generos": { "$size": 3 } } # exactamente 3 elementos
{ "reparto.0.actor": "Leonardo DiCaprio" } # acceso por índice
```

Subdocumentos (notación de punto)

```
{ "director.nacionalidad": "Británico" }
{ "plan.tipo": "premium" }
```

PARTE 5 — Defensa oral

Estas son las preguntas técnicas más probables en la defensa oral, organizadas por tema. Estudiá las respuestas hasta poder explicarlas con tus propias palabras sin leerlas. El profe puede reformular la pregunta — lo importante es entender el concepto, no memorizar la respuesta.

BLOQUE 1 — Modelado de datos

P: ¿Por qué usaste una colección separada para las reproducciones en lugar de embeberlas dentro del usuario?

Porque la relación entre usuarios y reproducciones es de crecimiento ilimitado. Un usuario activo puede acumular miles de reproducciones a lo largo del tiempo. Si las embebiéramos como un array dentro del documento del usuario, ese array crecería indefinidamente. MongoDB tiene un límite de 16 MB por documento, y eventualmente lo superaríamos. Además, cada vez que el usuario viera algo nuevo, habría que reescribir todo el documento del usuario para agregar un elemento al array — una escritura costosa e innecesaria. La solución correcta es la colección asociativa: cada reproducción es un documento independiente con referencias al usuario y a la película.

P: ¿Qué es una colección asociativa y cuándo se usa?

Es una colección que resuelve una relación N:M entre dos entidades que no tienen límite de crecimiento. En lugar de embeber una lista en alguno de los dos lados, creamos una colección separada donde cada documento representa una ocurrencia de esa relación. En nuestro proyecto, `reproducciones` es la colección asociativa entre `usuarios` y `peliculas`. Cada documento de reproducciones tiene `usuario_id`, `pelicula_id` y los datos propios de esa reproducción (progreso, fecha, si fue completada). Se usa cuando la relación crece sin límite previsible — en ese caso embeber en cualquiera de los dos lados viola la regla de expansión constante.

P: ¿Por qué el director está embebido dentro de la película y no en su propia colección?

Por tres razones concretas. Primera: la relación entre película y director es 1:1 — cada película tiene un único director embebido. Segunda: el dato es pequeño y de tamaño fijo — solo almacenamos nombre y nacionalidad. Tercera, y más importante: siempre accedemos al director junto con la película, nunca de forma independiente. No existe en nuestro sistema un caso de uso que requiera "dame todos los directores" sin el contexto de sus películas. Aplicando la heurística del espectro de crecimiento: el dato es Limitado y Pequeño, por lo tanto se embebe. Si necesitáramos consultar directores de forma independiente o si un director tuviera muchas propiedades complejas, la decisión cambiaría.

P: ¿Qué pasaría si modelaras esto de forma relacional, con tablas SQL?

En SQL tendríamos tablas separadas: `usuarios`, `peliculas`, `directores`, `generos`, `pelicula_genero` (tabla intermedia para la relación N:M), `reproducciones`. Para mostrar el detalle de una película con su director y géneros necesitaríamos al menos tres JOINS. En MongoDB, esa misma consulta es un único `find_one()` porque el director y los géneros ya están embebidos en el documento. La ventaja de SQL es la integridad referencial garantizada por el motor — si borro un director, la base de datos me avisa que hay películas que lo referencian. En MongoDB esa validación es responsabilidad de la aplicación. La ventaja de MongoDB es la velocidad de lectura cuando los datos que se necesitan juntos están almacenados juntos.

P: ¿Qué es el límite de 16 MB y cómo afectó tu diseño?

MongoDB tiene un límite de 16 MB por documento BSON. No es un límite que se alcance fácilmente con datos simples, pero es relevante cuando se embeben arrays que pueden crecer. En nuestro caso, fue el argumento técnico central para no embeber las reproducciones dentro del usuario: si un usuario viera 10.000 películas con metadata de cada reproducción, el documento podría superar ese límite. También fue el argumento para no embeber temporadas y episodios completos en el documento de una serie (en el modelo conceptual extendido). El límite nos obliga a pensar en el crecimiento futuro de cada array embebido antes de decidir embeber.

P: ¿Cómo resolviste la relación N:M entre usuarios y películas?

Con una colección asociativa llamada `reproducciones`. En un modelo relacional, una relación N:M se resuelve con una tabla intermedia que tiene las claves foráneas de ambas tablas. En MongoDB aplicamos el mismo concepto: cada documento de `reproducciones` tiene `usuario_id` (referencia a `usuarios`) y `pelicula_id` (referencia a `peliculas`), más los atributos propios de esa relación: cuántos minutos vio, si la completó, cuándo fue. La diferencia con SQL es que en MongoDB no hay integridad referencial automática — si borramos un usuario, sus reproducciones no se borran solas; tenemos que manejarlo desde la aplicación.

BLOQUE 2 — Aggregation Pipeline

P: Explicame el Pipeline 1 paso a paso.

El Pipeline 1 responde a la pregunta: ¿cuáles son las 5 películas más vistas? Parte de la colección `reproducciones`.

Primero, `$group` agrupa todos los documentos por `pelicula_id` y cuenta cuántos hay en cada grupo con `$sum`: 1. El resultado es un documento por película con su conteo de reproducciones.

Segundo, `$sort` ordena esos documentos de mayor a menor por el conteo.

Tercero, `$limit`: 5 se queda con los primeros 5. Este paso es clave: lo ponemos antes del `$lookup` deliberadamente, para que la etapa siguiente solo procese 5 documentos en lugar de todos.

Cuarto, `$lookup` une esos 5 documentos con la colección `peliculas` buscando el `_id` de la película. Agrega los datos de la película como un array llamado `info_pelicula`.

Quinto, `$unwind` convierte ese array de un elemento en un subdocumento plano.

Sexto, `$project` da forma al output final: excluye el `_id`, muestra el título, la calificación y el total de reproducciones.

P: ¿Por qué en el Pipeline 1 pusiste el \$lookup después del \$limit y no antes?

Por rendimiento. `$lookup` es la operación más costosa del pipeline porque tiene que buscar en otra colección. Si lo ponemos antes del `$limit`, haríamos el join sobre todos los documentos agrupados — que pueden ser cientos o miles de películas distintas. Si lo ponemos después del `$limit`, hacemos el join solo sobre los 5 documentos que ya sabemos que son el resultado final. El principio es: reducir el volumen de datos lo antes posible para que las etapas costosas trabajen sobre el menor conjunto posible.

P: ¿Para qué sirve \$unwind? ¿Por qué lo necesitás después del \$lookup?

`$unwind` toma un campo que es un array y genera un documento separado por cada elemento de ese array. Lo necesitamos después de `$lookup` porque esa etapa siempre devuelve los documentos

coincidentes como un array, incluso cuando hay un único resultado. Sin `$unwind`, el campo `info_pelicula` sería `[{ "titulo": "Inception", ... }]` — un array con un objeto adentro — y no podríamos acceder a `info_pelicula.titulo` directamente. Después del `$unwind`, ese campo pasa a ser `{ "titulo": "Inception", ... }` — un subdocumento plano al que sí podemos acceder con notación de punto.

También usamos `$unwind` en el Pipeline 2 sobre el array `generos`, que está embebido en cada película. Eso genera un documento por cada género por cada reproducción, lo que nos permite luego agrupar y contar cuántas veces aparece cada género.

P: ¿Cuál es la diferencia entre `find()` y el Aggregation Pipeline?

`find()` filtra y proyecta documentos existentes — el resultado siempre son documentos de la colección original, posiblemente con algunos campos incluidos u omitidos. El Aggregation Pipeline transforma documentos: puede calcular nuevos campos, agrupar por cualquier criterio, combinar colecciones y producir un resultado que no se parece en nada a los documentos originales. En términos SQL: `find()` es un `SELECT ... WHERE`; el pipeline es un `SELECT ... GROUP BY ... HAVING ... JOIN`. Usamos `find()` cuando solo necesitamos recuperar datos. Usamos el pipeline cuando necesitamos calcular, agrupar o combinar.

P: ¿Qué hace `$group` exactamente? ¿Qué es el campo `_id` en esa etapa?

`$group` agrupa los documentos que llegan a esa etapa según el criterio definido en `_id`. Todos los documentos que tengan el mismo valor de `_id` quedan en el mismo grupo, y sobre ese grupo se aplican los operadores de acumulación. Si `_id` es `"$pelicula_id"`, se forma un grupo por cada valor distinto de `pelicula_id`. Si `_id` es `null`, todos los documentos van al mismo grupo — útil para totales globales. Los operadores más usados son `$sum` para contar o sumar, `$avg` para promediar, `$first` para tomar el primer valor del grupo y `$push` para acumular valores en un array.

P: ¿Por qué en el Pipeline 3 el `$match` va después del `$lookup` y no antes?

Porque el campo que necesitamos para filtrar — `plan.tipo` — no existe en la colección `reproducciones`. Solo existe en la colección `usuarios`. Entonces, primero tenemos que traer los datos del usuario con `$lookup`, aplanarlos con `$unwind`, y recién ahí podemos filtrar con `$match` usando la notación de punto `"usuario.plan.tipo": "premium"`. Si pusiéramos el `$match` antes del `$lookup`, MongoDB buscaría el campo `plan.tipo` en `reproducciones` y no lo encontraría — el filtro no funcionaría. El orden de las etapas no es arbitrario: cada etapa trabaja con los documentos que le entrega la anterior.

P: ¿Qué es IXSCAN y COLLSCAN?

Son los dos modos en que MongoDB puede recorrer una colección para buscar documentos. COLLSCAN (Collection Scan) recorre todos los documentos de la colección uno por uno hasta encontrar los que cumplen el filtro — es $O(n)$ y se vuelve muy lento con colecciones grandes. IXSCAN (Index Scan) usa un índice para encontrar directamente los documentos que cumplen el filtro — es logarítmico y mucho más rápido. En el contexto del Aggregation Pipeline, si ponemos `$match` al principio del pipeline sobre un campo que tiene índice, MongoDB usa IXSCAN y reduce drásticamente el trabajo de las etapas siguientes. Si el campo no tiene índice o el `$match` no está al principio, hace COLLSCAN.

BLOQUE 3 — Python y pymongo

P: ¿Cómo se conecta Python a MongoDB? Explicame el código de `db.py`.

`db.py` tiene una función `get_db()` que devuelve el cliente de MongoDB y la base de datos. Usa `MongoClient` del driver oficial `pymongo` para establecer la conexión. La URI de conexión no está hardcodeada en el código: se lee desde una variable de entorno usando `python-dotenv`, que carga el archivo `.env`. Esto es importante por seguridad: si el código sube a GitHub, la URI no queda expuesta. La función devuelve una tupla (`client`, `db`) y el código que la llama es responsable de cerrar el cliente con `client.close()` cuando termina de operar.

P: ¿Por qué usaron un archivo `.env` en lugar de poner la URI directamente en el código?

Por seguridad y por buena práctica de desarrollo. Si ponemos la URI directamente en el código y ese código sube a un repositorio público (o incluso privado con colaboradores), las credenciales quedan expuestas. Con `.env`, las credenciales viven en un archivo local que está en el `.gitignore` y nunca se sube al repositorio. Además, si el entorno cambia — por ejemplo, si pasamos de MongoDB local a Atlas — solo hay que cambiar el `.env`, no el código.

P: ¿Qué devuelve `find()` en `pymongo`? ¿Qué es un cursor?

`find()` devuelve un objeto `Cursor`, no una lista. El `Cursor` es iterable y lazy: no trae todos los documentos a memoria de una vez, sino que los va cargando a medida que se itera. Esto es eficiente para colecciones grandes. Si necesitamos una lista con todos los resultados, usamos `list(cursor)` para materializarlo. En nuestro código usamos `list(db.coleccion.aggregate(pipeline))` por eso: para convertir el cursor del pipeline en una lista que podemos recorrer e imprimir.

P: ¿Qué hace `insert_many()` y qué devuelve?

`insert_many()` recibe una lista de documentos y los inserta todos en la colección en una sola operación de red — más eficiente que llamar a `insert_one()` en un bucle. Devuelve un objeto `InsertManyResult` que tiene el atributo `inserted_ids`: una lista con los `_id` de todos los documentos insertados, en el mismo orden en que fueron enviados.

BLOQUE 4 — Preguntas conceptuales generales

P: ¿Cuándo usarías MongoDB y cuándo SQL?

MongoDB es preferible cuando el esquema de datos es flexible o puede cambiar con el tiempo, cuando las lecturas son frecuentes y necesitamos que sean muy rápidas, cuando hay relaciones de alta cardinalidad que crecen sin límite, o cuando el volumen de datos requiere escalar horizontalmente. SQL es preferible cuando la integridad referencial es crítica y no puede depender de la aplicación, cuando necesitamos transacciones complejas con múltiples tablas, cuando los datos son altamente relacionales y las consultas ad hoc son frecuentes, o cuando el esquema es estable y bien definido desde el principio. No es que uno sea mejor que el otro — depende del caso de uso.

P: ¿Qué perderías si migraras tu proyecto a SQL?

Perderíamos velocidad de lectura en la pantalla de detalle de una película: en MongoDB es un solo `find_one()` que trae título, director y géneros juntos. En SQL serían varios JOINS. También perderíamos flexibilidad de esquema: si quisiéramos agregar un nuevo campo a algunas películas pero no a otras, en MongoDB simplemente lo agregamos; en SQL necesitaríamos una migración de la tabla. Ganaríamos integridad referencial garantizada por el motor y mayor facilidad para consultas ad hoc complejas que no estaban planificadas en el diseño original.

P: ¿Qué es el principio de "diseño impulsado por consultas"?

Es el enfoque central del modelado NoSQL: antes de definir las colecciones y sus estructuras, hay que identificar qué consultas va a necesitar el sistema y con qué frecuencia. La estructura del modelo se diseña para que esas consultas sean eficientes, no para que el esquema quede "normalizado" o "elegante". En la práctica significa preguntarse: ¿qué datos se leen siempre juntos? → embeberlos. ¿Qué datos crecen sin límite? → colección separada. Es lo opuesto al modelo relacional, donde primero se normaliza y después se optimizan las consultas.

P: Si tuvieras que agregar índices a tu proyecto, ¿sobre qué campos los pondrías y por qué?

Sobre los campos que más aparecen en `$match` y en las condiciones de búsqueda. En `reproducciones`, pondría un índice compuesto sobre (`usuario_id`, `pelicula_id`) porque el Pipeline 2 filtra por `usuario_id` y el Pipeline 3 hace un join por `usuario_id`. En `usuarios`, pondría un índice único sobre `email` para garantizar que no haya duplicados y acelerar la búsqueda por email (que sería el campo de login). En `peliculas`, dependiendo de los casos de uso, podría agregar un índice de texto sobre `titulo` para búsquedas full-text. El principio es: los índices aceleran las lecturas pero ralentizan las escrituras, porque MongoDB tiene que actualizar el índice cada vez que se inserta o modifica un documento.

BLOQUE 5 — Preguntas sobre el código específico

P: ¿Por qué en `seed.py` hacés `drop()` de las colecciones al principio?

Para garantizar que cada vez que ejecutamos el seed partimos de un estado limpio y conocido. Si no borráramos las colecciones antes, cada ejecución de `seed.py` agregaría más documentos encima de los anteriores, y los `_id` numéricos que usamos (1, 2, 3...) causarían errores de clave duplicada en la segunda ejecución. En un entorno de producción nunca haríamos esto — borrar datos reales es destructivo. Pero en un entorno de desarrollo y prueba, es la forma más simple de asegurar que los datos de ejemplo siempre son exactamente los que esperamos.

P: ¿Qué hace `{ "$first": "$usuario.nombre" }` en el Pipeline 3?

`$first` es un operador de acumulación que toma el valor del campo especificado del primer documento de cada grupo. Lo usamos en el Pipeline 3 porque después de agrupar por `usuario_id`, necesitamos el nombre del usuario para mostrarlo en el resultado. Todos los documentos del mismo grupo tienen el mismo nombre de usuario (porque agrupamos por su ID), así que tomar el primero es equivalente a tomar cualquiera. `$first` es la forma idiomática de "dame un valor representativo del grupo cuando sabés que todos los elementos tienen el mismo valor".

P: ¿Qué pasa si `MONGO_URI` no está definida en el `.env`?

En `db.py` usamos `os.getenv("MONGO_URI", "mongodb://localhost:27017")`. El segundo argumento es el valor por defecto: si la variable no está definida en el `.env`, Python usa `"mongodb://localhost:27017"` automáticamente. Esto hace al código más robusto: funciona tanto si el `.env` existe y define la variable, como si no existe (usando la conexión local por defecto). En un entorno de producción, se podría hacer más estricto y lanzar un error si la variable no está definida, para evitar conectarse accidentalmente a una base de datos equivocada.

PARTE 6 — Uso de IA en el desarrollo del proyecto

Herramienta utilizada

Durante el desarrollo de este proyecto utilizamos **Claude** (Anthropic, modelo Sonnet), accedido a través de la interfaz web de `claude.ai`. No utilizamos GitHub Copilot ni otras herramientas de autocompletado de código.

¿Para qué lo usamos concretamente?

El uso de la IA no fue uniforme a lo largo del proyecto. Distinguimos tres tipos de uso bien diferenciados:

1. Como tutor para comprender conceptos antes de implementar

Antes de escribir cualquier línea de código, utilizamos Claude para clarificar dudas conceptuales sobre el modelo de datos. Por ejemplo, consultamos explícitamente cómo aplicar la heurística del espectro de crecimiento (Limitado vs. Sin Límites) al dominio de una plataforma de streaming, y le pedimos que justificara cada decisión de embedding y referencing en términos de las reglas vistas en clase.

Este uso nos permitió tomar decisiones de diseño con fundamento teórico, no por intuición o por "lo que parecía correcto".

2. Como asistente para escribir y depurar código

Una vez que teníamos claras las estructuras de datos, le pedimos a Claude que nos ayudara a escribir los scripts de Python (`db.py`, `seed.py`, `queries.py`). En todos los casos, antes de aceptar el código generado:

- Lo leímos línea por línea para entender qué hacía cada parte.
- Lo ejecutamos localmente y verificamos que la salida coincidiera con lo esperado según los datos de ejemplo que nosotros mismos diseñamos.
- En algunos casos, detectamos que el pipeline necesitaba ajustes (por ejemplo, el orden de las etapas en el Pipeline 1 para que el `$limit` fuera antes del `$lookup`) y le pedimos que lo explicara antes de aceptar el cambio.

3. Como herramienta de documentación y estructuración

Usamos Claude para organizar el contenido del informe: la estructura de la Parte 2, la selección de ejemplos JSON representativos y la redacción técnica de las justificaciones de diseño. En todos los casos revisamos que la terminología usada coincidiera con la de los apuntes de clase (por ejemplo, que se mencionara explícitamente "colección asociativa", "expansión constante" y "diseño impulsado por consultas").

¿Cómo evaluamos si el código generado era correcto?

Aplicamos tres criterios de verificación:

Verificación funcional: ejecutamos `seed.py` y `queries.py` en nuestra instalación local de MongoDB y comparamos la salida con los resultados esperados que habíamos calculado manualmente de antemano. Por ejemplo, sabíamos que Inception debía aparecer primero en el Pipeline 1 con 3 reproducciones, porque contamos manualmente cuántas veces aparecía en los datos de ejemplo.

Verificación conceptual: para cada etapa de cada pipeline, nos aseguramos de poder explicar con nuestras palabras qué hace esa etapa y por qué está en ese lugar. Si no podíamos explicarlo, pedíamos una aclaración antes de seguir.

Verificación por consistencia con el material de clase: comparamos las decisiones de diseño sugeridas por la IA con lo que aparece en los apuntes y en los ejercicios de clase. Cuando hubo discrepancias de terminología, priorizamos la nomenclatura del curso.

¿Qué aprendimos del proceso?

Lo más valioso fue que el uso de la IA forzó a hacer explícitas las decisiones de diseño. Cuando le pedís a un modelo que justifique por qué eligió embedding en lugar de referencing, el proceso de leer esa justificación y evaluarla te obliga a entender el criterio, no solo aplicarlo mecánicamente.

Una limitación concreta que encontramos: en una versión inicial del Pipeline 3, Claude propuso usar `$match` antes del `$lookup` para filtrar usuarios premium. Eso era incorrecto, porque el campo `plan.tipo` no existe en la colección `reproducciones` — solo existe en `usuarios`. Detectar ese error nos hizo entender mejor el orden de ejecución del pipeline y por qué importa en qué colección está cada campo.

Otra limitación: la IA genera código que funciona, pero no siempre genera el código más eficiente. En el Pipeline 1, la primera versión hacía el `$lookup` antes del `$limit`, lo que implicaba unir toda la colección con `peliculas` antes de recortar a 5 resultados. Reconocer esa ineficiencia y corregirla fue un aprendizaje que surgió de analizar el código, no de aceptarlo sin más.

Síntesis

El uso de IA en este proyecto funcionó como un amplificador de aprendizaje cuando se usó con criterio: haciendo preguntas específicas, verificando los resultados y siendo capaces de justificar cada decisión. No reemplazó el proceso de comprensión — lo estructuró. Podemos defender cada línea de código y cada decisión de modelado porque las entendemos, no porque las copiamos.

PARTE 7 — Reflexión

P1 — ¿Qué fue lo más difícil de desaprender del modelo relacional?

Lo más difícil fue dejar de pensar en normalización como punto de partida. En SQL aprendimos que duplicar datos es siempre malo, que todo tiene que estar en su propia tabla y que las relaciones se resuelven con JOINS. Al principio, cuando veíamos que en MongoDB se "repite" información dentro de un documento —como el nombre del director dentro de cada película— pensábamos que estábamos haciendo algo mal. Costó entender que en NoSQL la duplicación controlada es una decisión de diseño deliberada, no un error.

P2 — ¿Hubo un momento en que algo "hizo click"?

Sí. El momento fue cuando entendimos la heurística del espectro de crecimiento: si la relación es limitada, embebés; si es ilimitada, colección separada. Antes de eso tomábamos las decisiones un poco a ojo. Cuando lo aplicamos al caso de las reproducciones —un usuario puede ver miles de películas, no podés embeber eso— todo el modelo empezó a tener lógica propia. Dejó de ser una regla arbitraria y se convirtió en algo que podíamos razonar.

P3 — ¿Qué habilidades se llevan?

La más importante no es técnica: es la de pensar el diseño desde las consultas hacia atrás, no desde las entidades hacia adelante. Antes modelábamos primero y después veíamos cómo consultar. Ahora preguntamos primero qué necesitamos leer y con qué frecuencia, y eso define la estructura. También nos llevamos el Aggregation Pipeline, que al principio parecía innecesariamente complejo pero terminó siendo la herramienta más poderosa de la materia.

P4 — ¿Qué cambiarías del cursado?

Más ejercicios con datos reales y volumen real. En clase los ejemplos tienen 5 o 10 documentos, y con esa escala da lo mismo si usás embedding o referencing. El problema de los 16 MB, la diferencia entre IXSCAN y COLLSCAN, el costo de un \$lookup sobre miles de documentos: todo eso se entiende mucho mejor cuando hay datos reales de por medio. También estaría bueno ver al menos un caso de migración desde SQL a MongoDB para entender en la práctica qué se gana y qué se pierde.

Ahora compilo todo en el archivo final:

Creó un archivo, ejecutó un comando, archivo leído

Creó un archivo, ejecutó un comando, archivo leído

Parte 7 lista. Cuando la modifiquen, las cosas más fáciles de personalizar son:

- **El momento del "click"** — si a alguno le pasó en un ejercicio concreto de clase, en el momento de ejecutar el seed y ver los datos, o al armar el primer pipeline, reemplazá esa parte con ese momento real
- **La dificultad inicial** — si alguno venía de otro lenguaje o no tenía tanto fondo en SQL, eso cambia el tono de la primera respuesta
- **Lo que cambiarían** — si tienen una crítica genuina al cursado (ritmo, herramientas, forma de explicar), es el lugar ideal para ponerla sin consecuencias porque no se evalúa

Fuentes: Apuntes de clase — Bases de Datos 2 — UTN Avellaneda (2025). MongoDB Aggregation Pipeline Documentation: <https://www.mongodb.com/docs/manual/aggregation/>